

MEDIATEK

MT8676 Hypervisor Secure Boot User Manual

版本: 1.0
出版日期: 2024-11-14

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-11-14	高峰	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 Secure Boot 4

 1.1 概述 4

 1.1.1 名词解释 4

 1.1.2 BootRom 5

 1.2 安全启动 5

 1.2.1 安全启动检查流程 5

 1.3 下载代理认证 (DAA) 8

 1.4 安全特性配置 8

 1.4.1 生成密钥对 8

 1.4.2 开启安全启动 10

 1.4.3 编译软件 12

 1.4.4 签名 First-Loader 12

 1.4.5 签名 FIT 格式镜像 12

 1.4.6 签名 Yocto 文件系统镜像 12

 1.4.7 签名 MTK 安卓格式镜像 13

 1.5 签名 DA 13

 1.6 Authfile 生成 14

附件一 附加条款 15

图片目录

图 1-1. 安全启动检查流程 7

图 1-2. dakey.h 10

表格目录

表 1-1. 名词解释 4

1 Secure Boot

1.1 概述

本文档旨在提供 MT8676 SoC 及其配套 SDK 中安全启动功能的概览。在该产品的设计和部署周期中，功能和特性可能会发生变化，这些变化将在本文档的后续版本中记录。此外，本文档还介绍了如何启用安全特性。

1.1.1 名词解释

表 1-1. 名词解释

缩略词	全称及释义
DA	Download Agent 下载代理 (包括 DA_BR 和 DA_PL)
DAA	Download Agent Authentication 下载代理认证
DRAM	Data Random Access Memory 动态随机存取内存
eFuse	Electronic Fuse 电子保险丝
eMMC	Embedded MultiMedia Card 嵌入式多媒体卡
EVB	Evaluation Board 开发板
FIT	Flattened Image Tree 扁平树镜像
LK2	Litter Kernel 2 轻量内核 2
NVM	Non-Volatile Memory 非易失性内存
ROM	Read-only Memory 只读存储器
SBC	Secure Boot Check 安全启动检查
SDK	Software Development Kit 软件开发包
SRAM	Static Random Access Memory 静态随机存取存储器
TEE	Trusted Execution Environment 可信执行环境
UART	Universal Asynchronous Receiver/Transmitter 通用异步收发器
USB	Universal Serial Bus 通用串行总线
USB DL	USB Download USB 下载

1.1.2 BootRom

BROM (BootROM) 是 SoC ROM 内的软件，无法修改。它是应用处理器执行的第一个软件。主要工作总结如下：

- 对 SoC 进行基础的硬件配置（例如设置 PLL, 时钟等），以便启动系统。
- 从启动设备 UFS/eMMC/NAND/...）加载第一阶段引导程序以启动系统。
- 与主机 PC 端的工具通信，以加载下载代理（DA）软件，用于为没有软件或软件损坏的外部 Flash 进行镜像下载。
- 使用多个安全启动检查密钥，对第一阶段引导程序或下载代理进行安全启动检查。

1.2 安全启动

1.2.1 安全启动检查流程

安全启动检查流程用于从硬件信任根创建可信执行链。可以通过烧录 eFuse 中的 SBC_EN 来启用它。每次系统从上电复位中恢复时都会执行此流程。图 1-1 展示了基本的安全启动检查流程。详细描述如下。

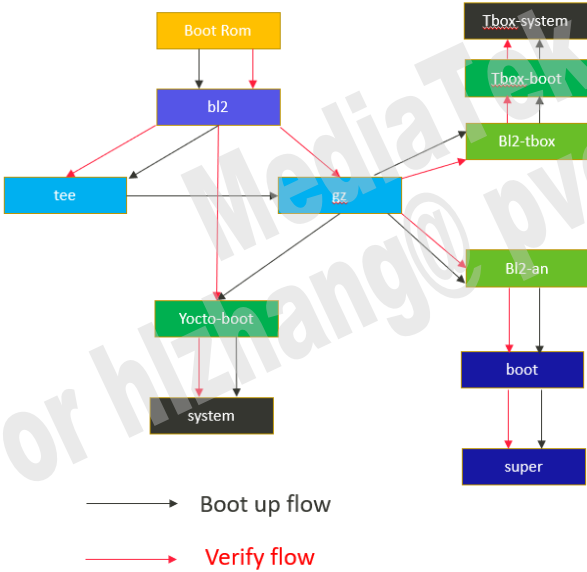
1. 上电复位后，BROM 使用以下流程来验证非易失性存储器（NVM，例如 eMMC 或 NAND）中的安全启动检查（SBC）公钥（SBC_PUBK）。BROM 依次使用来自 eFuse 的公钥 HASH（SBC_PUBK_HASH）。
 - (1) 从 eFuse 读取安全启动检查公钥（SBC_PUBK）的 HASH 值（SBC_PUBK_HASH）。
 - (2) 从 NVM 读取 SBC 公钥（SBC_PUBK）。
 - (3) 计算来自 (2) 的数据的 HASH 值。
 - (4) 检查来自 (1) 和 (3) 的数据是否相同。
2. BROM 加载并验证 bl2.img（Yocto LK2），这是第一阶段的引导加载程序。首先，BROM 将 Yocto LK2 从 NVM 读取到 SoC 的 SRAM 中，并使用安全启动检查公钥（SBC_PUBK）来认证 Yocto LK2。如果 Yocto LK2 验证成功，将执行 Yocto LK2。验证方法使用 SHA256 计算 HASH 值，RSA（2048 位）和 MTK 或 PSS 填充（由第一阶段引导加载程序的头部决定）来进行验证。

注意：

 - 如果外部存储器是 NAND 闪存，BROM 支持第二份（副本）LK2。当 BROM 无法加载第一份（副本）LK2 时，BROM 会尝试加载/认证第二份（副本）LK2。
3. 在第一阶段引导加载程序之后，这时跑在 Yocto LK2 阶段，LK2 会校验 Yocto FIT 镜像，Hypervisor 以及安卓相关的镜像。

4. Yocto LK2 从 NVM 把 FIT 镜像（比如 yocto 内核等 FIT 镜像）加载程序加载到非安全的 DRAM 区域并验证。它使用嵌入在 LK2 中的 VERIFIED 公钥来认证校验 FIT 镜像的 FIT 签名，并检测 FIT 镜像中各个子镜像的 HASH 值。验证方法使用 SHA256 计算 HASH 值，RSA（2048 位）和 MTK 或 PSS 填充进行验证
5. Yocto LK2 还从 NVM 加载 MTK 签名格式的镜像（比如 TEE，Hypervisor 镜像）到安全的 DRAM 区域并验证。它使用嵌入在 LK2 程序内的 ROOT 公钥来认证镜像的 CERT1，并使用从 CERT1 中提取的 IMAGE 公钥来认证镜像的 CERT2。如果 CERT2 验证成功，会认为此镜像合法，会继续跑后面的流程。验证方法使用 SHA256 计算 HASH 值，RSA（2048 位）和 MTK 或 PSS 进行验证。
6. Yocto LK2 在校验完上述 4 和 5 所需要校验的镜像后，会跳转到 TEE，TEE 会启动 Hypervisor 镜像 gz.img，Hypervisor 会先启动 Yocto 内核，然后再校验启动安卓 LK2 镜像和 Tbox LK2 镜像。
7. Yocto 内核执行后，可以通过 Yocto 内核中，提供的 root_check 功能来实现对只读系统的抽样校验。
8. Hypervisor 校验安卓 LK2 和 Tbox LK2 的方法需要 Hypervisor 的厂商来提供。目前是谦川。
9. 安卓 LK2 会通过 AVB 校验安卓内核，在安卓 Linux 系统执行后，可以通过 Linux 内核中的 dm-verity（在块设备上）来验证只读镜像。
注意：
 - 目前 dm-verity 仅支持 Linux 中的块设备（eMMC 上的 EXT4）。
10. Tbox LK2 会加载 Tbox 内核镜像到 DRAM 区域进行验证，Tbox 内核镜像 FIT 格式的镜像。Tbox LK2 会使用嵌入在 Tbox LK2 的 VERIFIED 公钥来认证校验 FIT 镜像的 FIT 签名，并检测 FIT 镜像中各个子镜像的 HASH 值。验证方法使用 SHA256 计算 HASH 值，RSA（2048 位）和 MTK 或 PSS 填充进行验证。
11. Tbox 内核执行后，可以通过 Yocto 内核中，MediaTek 提供的 root_check 功能来实现对只读系统的抽样校验。

Secure boot verify flow



MEDIATEK

MediaTek Proprietary and Confidential. © 2024 MediaTek Inc. All rights reserved.

	key: rsa2048 + sha256 key name: root key and image key 校验的方式: SBC public key的hash指会被存储到efuse中, Boot Rom会用这个key来校验Yocto LK2
	key: rsa2048 + sha256 key name: Yocto image key 校验的方式: Yocto boot.img 是采用的fit boot sign和 verify的方式, public key会存在LK2里面, 在Yocto Lk2 load的时候进行校验
	key: rsa2048 + sha256 key name: Yocto image key 校验的方式: Yocto OS是采用mtb verify的方式进行校验, public key会被存储到Yocto boot.img 里面
	key: rsa2048 + sha256 key name: Android lk2 verify key 校验的方式: Android LK2会采用rsa2048 + sha256的方法生成signature, public key会存在hypervisor里面对Android LK2进行校验(需要谦川帮忙)
	key: rsa2048 + sha256 key name: root key and image key 校验的方式: MTK Android secure boot sign + verify flow, 可以参考文档: 《Secure 2.1 Configuration SOP.docx》, android build 出来image: tee.img gz.img 安卓签名格式modem.img.img 等
	key: rsa2048 + sha256 key name: DM verify key 校验的方式: Android原生的DM verify flow, public key会存在Android boot.img中

图 1-1. 安全启动检查流程

1.3 下载代理认证 (DAA)

当外部存储器 (eMMC/NAND/...) 中的设备软件为空或损坏时, 主机 PC 工具可以与 SoC 中的 BROM 通信, 将一个 DA 软件加载到 SoC 的 SRAM 中并执行 DA 来进行镜像下载过程。当 Enable_DAA 被烧录时, DA 将由 BROM 进行认证, 下面的流程被称为下载代理认证 (DAA)。

1. 主机 PC 工具通过 USB 或 UART 将认证文件 (AuthFile) 发送给 BROM。
2. BROM 对 AuthFile 进行认证。
 - (1) BROM 从 AuthFile 中读取安全启动检查 (SBC) 公钥 (SBC_PUBK), 并使用公钥 HASH (SBC_PUBK_HASH~SBC_PUBK_HASH1) 及相应的禁用位 (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) 来认证 SBC 密钥的 HASH。
 - (2) BROM 使用 SBC_PUBK 来验证 AuthFile。
 - (3) BROM 从 AuthFile 中获取 DAA 密钥。
3. 主机 PC 工具发送 DA。
4. BROM 使用 DAA 密钥通过 SHA256/RSA-2048 验证 DA。
5. BROM 跳转到 DA 来执行固件下载过程。

注意:

- SBC_PUBK 和 DAA 密钥可以不同, 以提高安全级别。
- 除了 BROM, 第一阶段引导加载程序也支持下载代理协议, 它可以使用 SHA256/RSA2048 对 DA 进行验证。

1.4 安全特性配置

1.4.1 生成密钥对

Yocto 生成两对密钥 (包括私钥和公钥), SBC_KEY 和 VERIFIED_KEY, 并放到 Yocto 系统 `meta/meta-mediatek/conf/machine/keys` 目录下。

1. 生成 Yocto 私钥的命令:

```
openssl genrsa -F4 -out sbc_key.pem 2048
openssl genrsa -F4 -out verified_key.pem 2048
openssl req -batch -new -x509 -key verified_key.pem -out verified_key.crt
```

2. 生成 Yocto 公钥的命令:

```
openssl rsa -in sbc_key.pem -pubout > sbc_pubk.pem
openssl rsa -in verified_key.pem -pubout > verified_pubk.pem
```

安卓也生成两对密钥（包括私钥和公钥），根密钥 ROOT_KEY 和镜像密钥 IMAGE_KEY，使用 pem_to_der.py，在 vendor/mediatek/proprietary/scripts/sign-image_v2/der_extractor/ 中将根密钥的格式转换为 DER 格式。

1. 生成安卓私钥的命令:

```
openssl genrsa -out root_prvk.pem 2048
python pem_to_der.py root_prvk.pem root_prvk.der
openssl genrsa -out img_prvk.pem 2048
```

2. 生成安卓公钥的命令:

```
openssl rsa -in root_prvk.pem -pubout > root_pubk.pem
python pem_to_der.py root_pubk.pem root_pubk.der
openssl rsa -in img_prvk.pem -pubout > img_pubk.pem
```

再继续使用相同的方法生成用于签用于签名和验证 DA 的 DA 密钥对（da_prvk.pem/da_pubk.pem）。另外我们建议可以将 Yocto 私钥内容覆盖掉安卓私钥内容，Yocto 公钥内容覆盖掉安卓公钥内容，这样配置会更简化。

1.4.1.1 生成 oemkey.h

将根密钥（root_pubk.der）导出，使用 der_extractor 生成 oemkey.h，该工具位于

vendor/mediatek/proprietary/scripts/sign-image_v2/der_extractor/ 目录下。请将 oemkey.h 放置到以下路径:

Android DA: \$DA_Kit/Raphael-da/custom/\$PLATFORM/oemkey.h

Yocto LK2: \$LK/target/\$PROJECT/inc/oemkey.h

命令:

```
chmod 777 der_extractor
./der_extractor root_pubk.der oemkey.h ANDROID_SBC
```

1.4.1.2 生成 dakey.h

dakey.h 包含用于由 LK2 验证 DA_BR.bin 的 DA_BR 公钥。并使用相应的 DA 私钥对 DA_BR.bin 进行签名。

命令:

```
chmod 777 der_extractor
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

将 dakey.h 放到以下路径:

Yocto LK2: \$LK2/target/\$PROJECT/include/dakey.h

注意:

- 生成 dakey.h 后, 请将“OEM” 替换成 “DA” (图 1-2).

```
#ifndef _DAKEY_
#define _DAKEY_
#define DA_PUBK_SZ 256
#define DA_PUBK {0xD1, 0x64, 0x03, 0x46, 0x6C, 0x
0xE3, 0x32, 0xE1, 0x7D, 0xC0, 0
0x4E, 0xAA, 0xC2, 0xCB, 0x30, 0
0x68, 0x20, 0x0D, 0xE1, 0x7D, 0
0x73, 0x70, 0x50, 0xF0, 0x84, 0
0x0B, 0xB0, 0xCD, 0x60, 0x68, 0
0x71, 0x5C, 0x6B, 0x6B, 0x67, 0}
```

图 1-2. dakey.h

1.4.2 开启安全启动

1.4.2.1 安卓软件安全配置

将安卓侧软件安全配置默认都开启的, 可以检查下如下配置:

1. LK2 设定软件 (Hypervisor 安卓的 LK2 是在 Yocto 编译出来的)

Yocto LK2:

```
$LK/project/${PROJECT}.mk
MTK_SECURITY_SW_SUPPORT=yes
```

2. Kernel 设定

32-bit Kernel

```
<kernel path>/arch/arm/configs/<project>_debug_defconfig
<kernel path>/arch/arm/configs/<project>_defconfig
```

64-bit Kernel

```
<kernel path>/arch/arm64/configs/<project>_debug_defconfig
<kernel path>/arch/arm64/configs/<project>_defconfig
```

```
CONFIG_MTK_SECURITY_SW_SUPPORT=y
```

1.4.2.2 Yocto 侧软件安全配置

```
meta/meta-mediatek-mt8xxx/conf/machine/[project].conf
SECURE_BOOT_ENABLE = "yes"
ENABLE_ROOTFS_CHECK= "yes"
```

1.4.2.3 TBox 侧软件安全配置

```
meta/meta-mediatek-mt8xxx/conf/machine/[project]uos_tbox.conf
SECURE_BOOT_ENABLE = "yes"
ENABLE_ROOTFS_CHECK= "yes"
```

1.4.2.4 BROM 开启安全配置

前面三点都是开启软件层面的安全校验配置，开启 BROM 校验 first-loader（本项目 first-loader 就是 Yocto LK2）则需要写 eFuse 栏位。

1. 生成 SCB_PUBK HASH:

工具路径: meta/meta-mediatek/recipes-bsp/lk/files/pbp

命令:

```
chmod 777 der_extractor
python pbp.py -j sbc_key.pem -func keyhash_pss -o keyhash
```

2. 写 SBC_PUBK HASH 写进 eFuse:

Step 1: 使用 “hexdump -C keyhash”, or “xxd -c 32 keyhash” 来显示 16 进制 keyhash.

注意: 你必须使用 -c 这个入参才能生成规范的十六进制的 ASCII 显示结果.

```
hexdump -C keyhash
00000000 16 b1 0e fc 5e 4e 06 76 e9 d9 6e 40 0c 51 ca 36 | .....
00000010 d1 be 93 d2 67 fd 3e af db f6 f7 89 4a 2c 40 18 | .....
xxd -c 32 keyhash
00000000 16b1 0efc 5e4e 0676 e9d9 6e40 0c51 ca36 d1be 93d2 67fd 3eaf dbf6 f789 4a2c
4018      .....
```

禁止使用 “hexdump keyhash”来显示

```
hexdump keyhash
00000000 b116 fc0e 4e5e 7606 d9e9 406e 510c 36ca
00000010 bed1 d293 fd67 ad3e f6db 89f7 2c4a 1840
```

Step 2: 将结果转化为十六进制的字符串; 删除字符之间的空格:

```
16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

Step 3: 执行 ewriter 命令:

```
ewriter 1 0 32 16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

“ewriter”工具更详细的用法, 可以参考 MTK_eFuse_Writer_User_Guide.

注意:

- 开启 IC 的 BROM secure boot 校验功能, 除了要写上述的 SBC_PUBKO_HASH 栏位外, 还需要写 SBC_EN。eFuse 只能写一次, 所以在项目开发阶段不建议写 eFuse 开启 IC 的 BROM secure boot 校验功能。

1.4.3 编译软件

编译整个项目。

1.4.4 签名 First-Loader

1. 签名 first-loader (本项目是 Yocto LK2) 会用到 sbc_key.pem 和 verified_key.pem 两把 key
2. Key 放置的目录:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
3. 编译 Yocto LK2 时, 开启了安全配置会自动对 Yocto LK2 进行签名。
4. 签名过程: 会将 sbc pub key 和 verified pub key 打包进 key cert, 使用 sbc priv key 签名; 计算镜像 HASH, 将 HASH 和 verified pub key 打包进 content cert, 并使用 verified priv key 签名; 最后将 key cert 和 content cert 拼接到原始镜像尾部就生成了签名镜像。

1.4.5 签名 FIT 格式镜像

1. 签名 first-loader (本项目是 Yocto LK2) 会用到 verified_key.pem 这把 key
2. Key 放置的目录:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
3. 编译各 FIT 镜像时, 开启安全配置会自动对相应的镜像进行签名。
4. 签名过程: FIT 镜像时开源标准格式的镜像, 所以会使用 uboot-mkimage 工具对镜像进行签名, 主要是对原始计算 HASH 存入 HASH 节点, 并使用 verified priv key 对 FIT 镜像中除 data 节点以外的节点拼接后进行签名, 将签名的结果保存在 Signature 节点。

1.4.6 签名 Yocto 文件系统镜像

1. 签名 Yocto 文件系统也会用到会用到 verified_key.pem 这把 key
2. Key 放置的目录:
`<yocto branch>/meta/meta-mediatek/conf/machine/keys/`
3. 编译各文件系统镜像时, 开启安全配置会自动对相应的镜像进行签名。

4. 签名过程：MediaTek 会提供文件系统的签名工具 `mtd_verify`。此工具会根据输入的参数，从文件系统原始镜像中抽出 N 份数据，每份固定 size 和固定间隔；将抽出的部分拼接在一起计算 HASH，使用 verified 私钥对 HASH 进行签名；将输入参数的信息和 Signature 保存在原始镜像尾部。

1.4.7 签名 MTK 安卓格式镜像

1.4.7.1 生成 cert1 和 cert2 密钥

1. 使用 `root_prvk.pem` 和 `img_prvk.pem` 生成 `cert1` 和 `cert2_key`。
2. 运行以下命令。

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/img_key_deploy.py mt6897
cert1_key_path=${KEY_PATH}/root_prvk.pem cert2_key_path=${KEY_PATH}/img_prvk.pem
root_key_padding=pss 2>&1 | tee SecureGen.log
```

注意:

- 请不要进入 `./vendor/mediatek/proprietary/scripts/sign-image_v2/` 目录执行 `img_key_deploy.py`。请在代码库的根目录下执行该命令。
- 请检查所有镜像的 `cert1` 和 `cert2_key` 是否已在以下位置更新
vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert1\
vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert2_key\

1.4.7.2 签名镜像

在生成了 `cert1` 和 `cert2_key` 之后，您可以运行签名脚本来生成 `<image>-verified.bin` 或 `<image>-verified.img`。

命令:

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/SignFlow.py <platform> <project>
2>&1 | tee signflow.log
```

或者

```
./vendor/mediatek/proprietary/scripts/sign-image/sign_image.sh 2>&1 | tee signflow.log
```

(必须在编译前 lunch project)

1.5 签名 DA

1. 密钥路径设置

将 DA 公钥 (da_prvk.pem) 导出，放置到 Android 路径

vendor/mediatek/proprietary/scripts/secure_chip_tool/custom_keys 目录下

2. 将需要签名的 DA 放置在 prebuilt/resignda/ 目录下，并执行以下命令来签名 DA：

```
cd vendor/mediatek/proprietary/scripts/secure_chip_tool/  
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897  
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

1.6 Authfile 生成

DA 的公钥包含在 authfile 中，BROM 使用它来验证 DA。因此，如果启用了 DAA，在使用 flashtool 下载镜像时需要 authfile。所有位于以下位置的 .ini 文件，已经为本项目配置好了。

./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/

您只需要替换位于以下位置的 DA、和根私钥的 .pem 文件。

./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/

Authfile 生成命令：

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g  
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。